

Projekt SWTM

Dokumentation

Aktivitätstracker

Android Application

Auftraggeber: pep.digital GmbH, Esslingen

Modul: Projektarbeit SWTM

Hochschule: Hochschule Esslingen

Semester: Sommersemester 2026

Abgabedatum: 10. April 2026

Teammitglieder

Name	Matrikelnummer
Daniel Trbara	779034
Christian Sakru	779980
Lionel Toussoumsnaba	781030
Mirac Sancak	781232
Abdullah Bilen	779202

Inhaltsverzeichnis

1	Projektübersicht	1
1.1	Problemstellung und Zielgruppe	1
1.1.1	Problemstellung	1
1.1.2	Zielgruppe	1
1.2	Geforderter Funktionsumfang	2
1.2.1	Minimum Viable Product (MVP)	2
1.2.2	Erweiterte Ziele (Extended Goals)	2
1.2.3	Zielsetzung	3
1.3	UI-Entwürfe	3
1.4	Architektur (konzeptionell + technologisch)	11
1.4.1	Logische Sicht	11
1.4.2	Technischer Stack	12
1.4.3	Begründung der Technologiewahl	12
1.5	Projektmanagement	13
1.5.1	Ziel des Projektmanagements	13
1.5.2	Arbeitsweise	13
1.5.3	Kommunikations- und Organisationstools	14
1.6	Aufwandsschätzung	14
	Anhang	15
	Quellen	15

Meilenstein 1 Projektübersicht

Dieses Dokument beschreibt die erste Fassung der Projektdokumentation des **Aktivitätstrackers** im Rahmen des Moduls Projektarbeit SWTM an der Hochschule Esslingen, Sommersemester 2026. Es wurde im Auftrag der *pep.digital GmbH* (Esslingen) erstellt und umfasst Problemstellung, Zielgruppe, geforderter Funktionsumfang, UI-Entwürfe, Architektur, Projektmanagement sowie eine erste Aufwandsschätzung des Projekts.

1.1 Problemstellung und Zielgruppe

1.1.1 Problemstellung

Viele Menschen verbringen einen Großteil ihres Alltags sitzend, sei es im Büro oder im Home-Office vor dem PC. Bewegung kommt dabei häufig zu kurz. Vor allem in den Wintermonaten verstärkt sich dieses Problem durch den Mangel an Sonnenlicht. Langes Sitzen ohne ausreichende Bewegung führt zu einer schlechten Körperhaltung, die langfristig die Muskulatur schwächt.

Kernproblem: Büroangestellte und Home-Office-Arbeitende bewegen sich im Alltag zu wenig, was zu gesundheitlichen Langzeitfolgen führt. Es fehlt ein niedrigschwelliges, motivierendes Werkzeug zur Integration von Alltagsbewegung.

Hier setzt der **Aktivitätstracker** an: Er soll Nutzerinnen und Nutzer motivieren, regelmäßig Outdoor-Aktivitäten durchzuführen. Dies wird durch ein Streak-System und tagesbasierte Erinnerungen unterstützt.

1.1.2 Zielgruppe

Die Anwendung richtet sich primär an folgende Nutzergruppe:

- Personen, die überwiegend sitzend im Büro oder Home-Office arbeiten
- Nutzer, die fit und gesund bleiben möchten, mit leichter regelmäßiger Outdoor-Aktivität (z. B. Spazierengehen, Radfahren)
- Die Zielgruppe ist **nicht** leistungsorientiert

1.2 Geforderter Funktionsumfang

1.2.1 Minimum Viable Product (MVP)

Das MVP beschreibt die grundlegenden Funktionen, die notwendig sind, um einen ersten nutzbaren Prototyp bereitzustellen.

Funktion	Beschreibung
Aktivitäts-Tracking	Nutzer können tägliche Aktivitäten (z. B. Gehen, Radfahren) erfassen.
Streak-System	Das System zählt aufeinanderfolgende aktive Tage. Bei fehlender Aktivität wird die Streak unterbrochen. Dient als Motivationsmechanismus.
Widgets	Home-Screen-Widgets ermöglichen schnelle Eingabe und direkte Streak-Anzeige ohne App-Öffnung.
Wetter/tagesbasierte Erinnerungen	Push-Benachrichtigungen basierend auf Uhrzeit oder Wetterbedingungen motivieren zur Aktivität bei günstigen äußeren Bedingungen.

1.2.2 Erweiterte Ziele (Extended Goals)

Die folgenden Funktionen erweitern den MVP und verbessern die Benutzererfahrung. Sie sind für die erste Version nicht zwingend erforderlich.

- **Statistiken:** Längste Streak, Wochen-/Monatsübersichten, Aktivitätsanalysen
- **Dark-/Light-Theme:** Umschaltbares Design nach Nutzerpräferenz
- **Mehrsprachigkeit:** Deutsch und Englisch als primäre Sprachen
- **Streaks mit Freunden teilen:** Soziale Komponente zur gegenseitigen Motivation
- **Öffentliche Challenges:** Gruppenherausforderungen zur Steigerung der Motivation durch Wettbewerb

1.2.3 Zielsetzung

Der Aktivitätstracker verfolgt konkrete, messbare Ziele, die über die allgemeine Problemlösung hinausgehen und als Erfolgskriterien für das Projekt dienen:

- Nutzer sollen jeden Tag eine Outdoor-Aktivität erfassen können
- Eine Aktivität soll innerhalb von wenigen Sekunden erfassbar sein
- Wetter- und tagesbasierte Benachrichtigungen sollen automatisch bei geeigneten Bedingungen ausgelöst werden
- Das Streak-System soll die tägliche Nutzungsfrequenz messbar erhöhen, ohne dabei leistungsorientierten Druck aufzubauen

Auf Basis dieser Ziele wurden die folgenden Architektur- und Technologieentscheidungen getroffen, die in Abschnitt [1.4](#) beschrieben werden.

1.3 UI-Entwürfe

Die folgenden Abbildungen zeigen die Entwürfe der Anwendung. Die Bildunterschriften sind jeweils mit dem Quellenverzeichnis verlinkt. Alle UI-Entwürfe wurden von Lionel Toussoumsnaba in Figma entwickelt (siehe Quelle [\[1\]](#)).

Der Hauptbildschirm (siehe Abbildung 1.1) zeigt die aktuelle Streak-Anzeige des Nutzers. Die Streak-Visualisierung ist von modernen Tracking-Apps wie YAZIO oder Duolingo inspiriert.

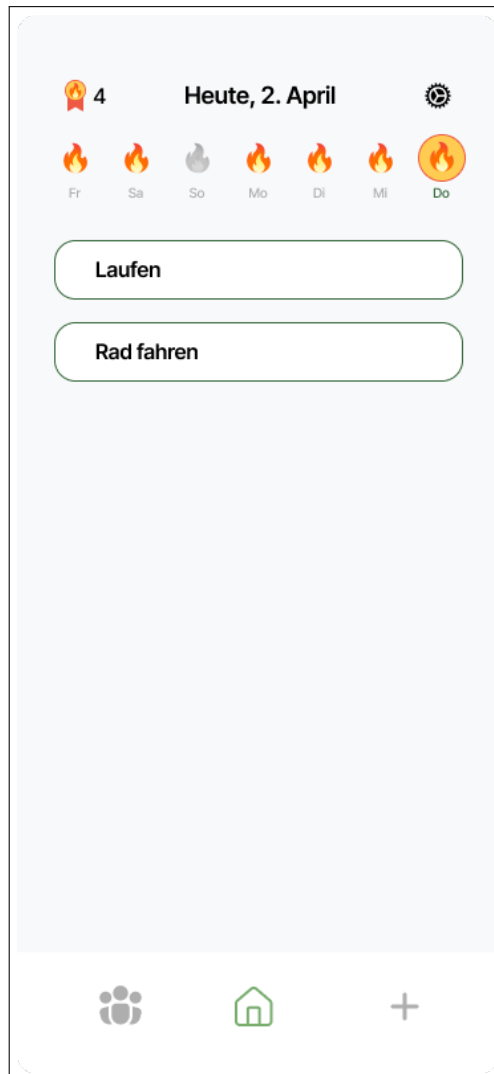


Abbildung 1.1: [Home Screen: Hauptmenü mit Streak-Anzeige](#)

Die Registrierung (siehe Abbildung 1.2) ermöglicht die erstmalige Anmeldung per E-Mail oder alternativ über ein Google-Konto. Der Anmeldebildschirm (siehe Abbildung 1.3) bietet dieselben Login-Optionen für bereits registrierte Nutzer.

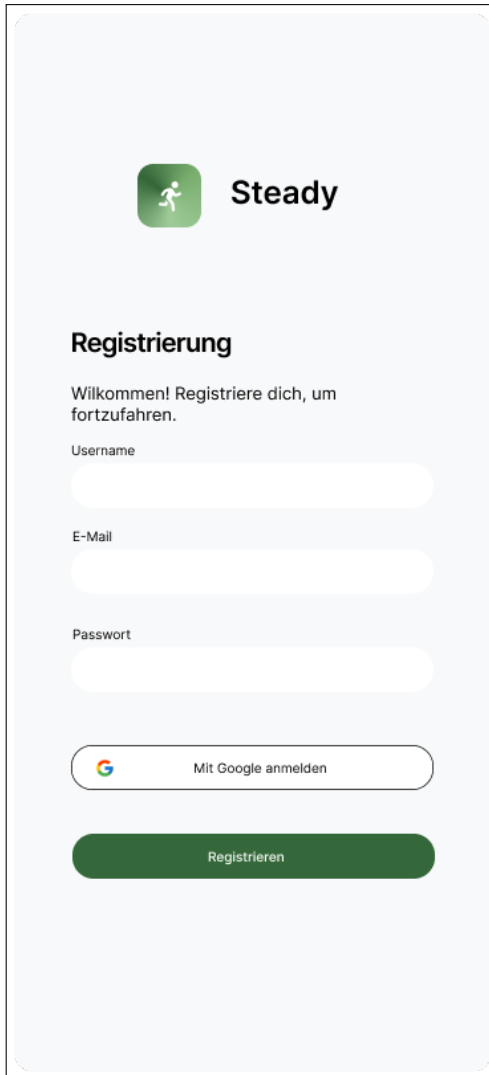
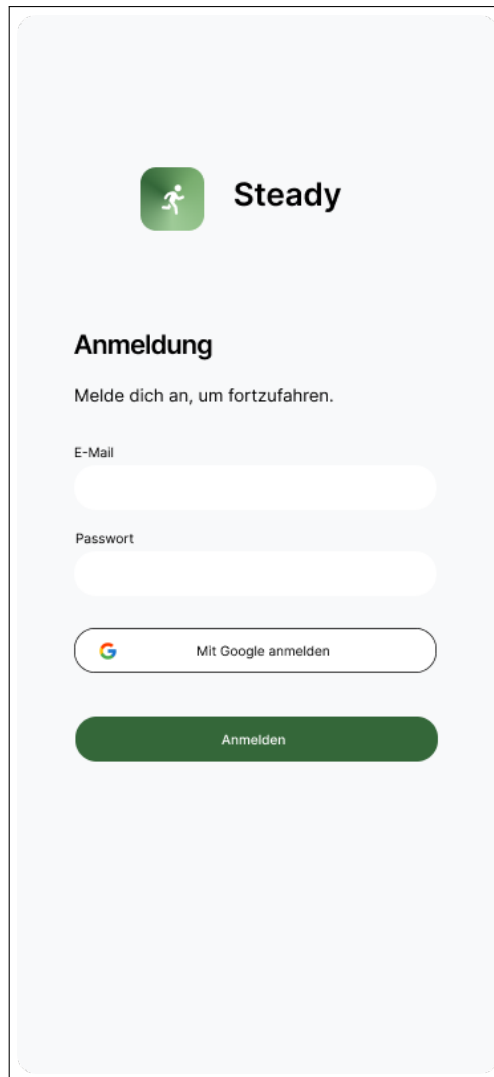
The image shows a mobile app registration screen for 'Steady'. At the top, there is a green square icon with a white running figure and the word 'Steady' in bold black text. Below this, the heading 'Registrierung' is displayed in bold. A welcome message reads: 'Willkommen! Registriere dich, um fortzufahren.' There are three input fields: 'Username', 'E-Mail', and 'Passwort', each with a light gray label and a white rounded rectangular input area. Below the password field is a button with the Google logo and the text 'Mit Google anmelden'. At the bottom is a large green button with the text 'Registrieren' in white.

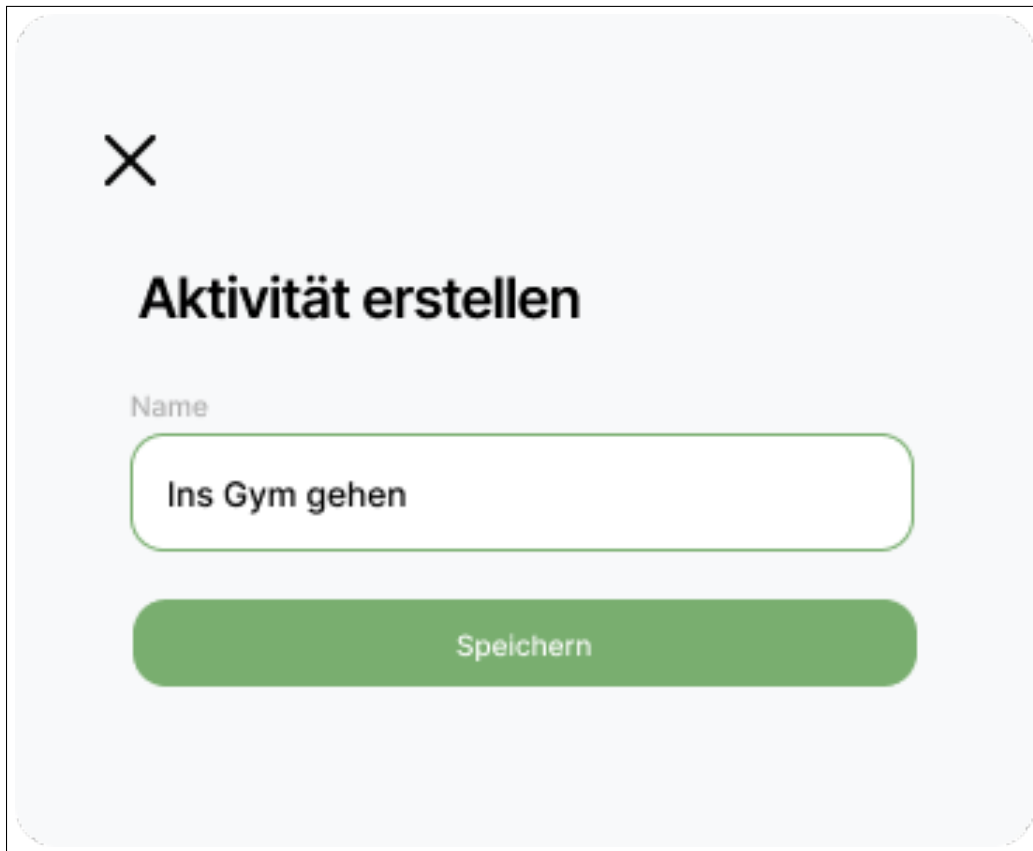
Abbildung 1.2: Registrierung: Übersicht zur erstmaligen Anmeldung oder alternatives Google-Login



The image shows a mobile application login screen for 'Steady'. At the top, there is a green square icon with a white running figure, followed by the word 'Steady' in a bold, black, sans-serif font. Below this, the heading 'Anmeldung' is displayed in bold black text. Underneath the heading is the instruction 'Melde dich an, um fortzufahren.' in a smaller, regular black font. The login form consists of two white input fields with rounded ends. The first field is labeled 'E-Mail' and the second is labeled 'Passwort'. Below these fields is a button with a white background and a thin black border, featuring the Google logo on the left and the text 'Mit Google anmelden' on the right. At the bottom of the form is a solid green button with the text 'Anmelden' in white.

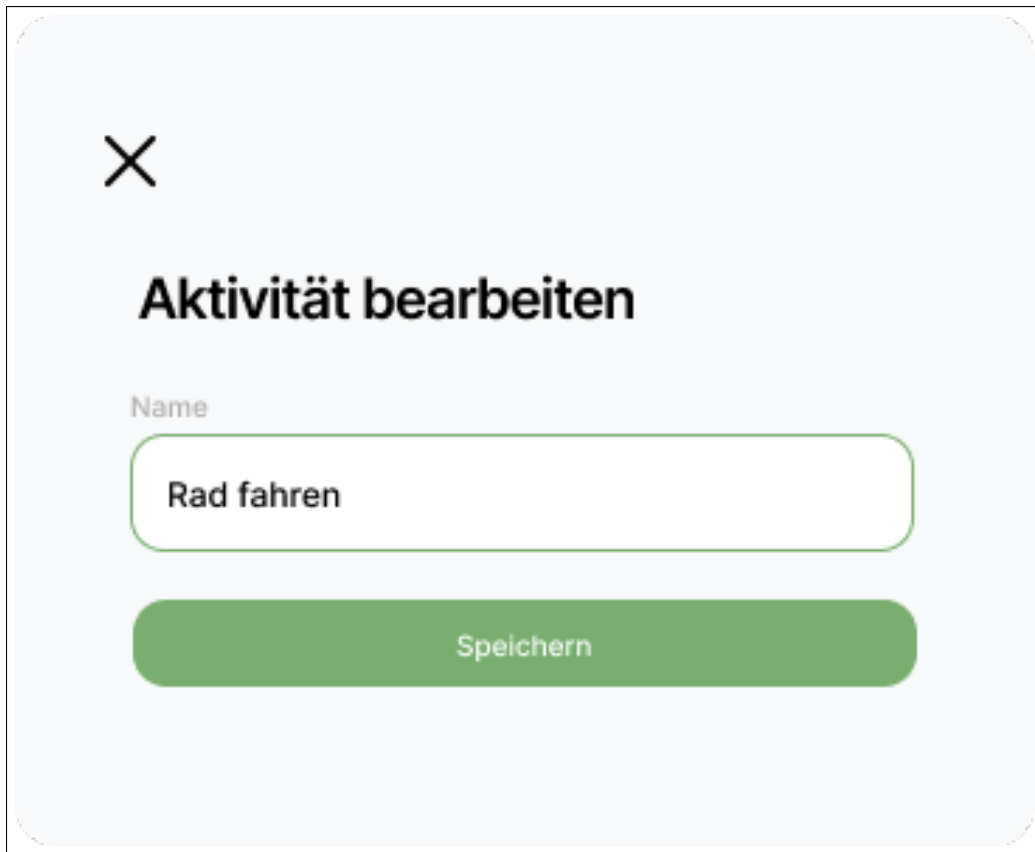
Abbildung 1.3: Anmeldebildschirm: Login via E-Mail und Passwort oder alternatives Google-Login

Neue Aktivitäten können über ein Modal erfasst werden (siehe Abbildung 1.4). Bestehende Einträge lassen sich anschließend über einen separaten Bearbeitungsscreen anpassen (siehe Abbildung 1.5).



The image shows a modal window titled "Aktivität erstellen" (Create Activity). It features a close button (X) in the top left corner. Below the title is a label "Name" followed by a text input field containing the text "Ins Gym gehen". At the bottom of the modal is a green button labeled "Speichern" (Save).

Abbildung 1.4: Aktivität erstellen: Modal zur Eingabe und Speicherung einer neuen Aktivität



A modal dialog box with a light gray background and rounded corners. In the top-left corner, there is a large black 'X' icon. Below it, the title 'Aktivität bearbeiten' is displayed in a bold, black, sans-serif font. Under the title, the label 'Name' is shown in a smaller, gray font. Below the label is a white text input field with a green border, containing the text 'Rad fahren'. At the bottom of the dialog is a wide, green button with rounded corners and the text 'Speichern' in white.

Abbildung 1.5: [Aktivität bearbeiten](#): Modal zur Anpassung einer bereits bestehenden Aktivität

Im Einstellungsbereich (siehe Abbildung 1.6) können Nutzer Benachrichtigungszeiten und Kontodetails verwalten. Die Profileinstellungen (siehe Abbildung 1.7) ermöglichen zusätzlich die Änderung des Benutzernamens.

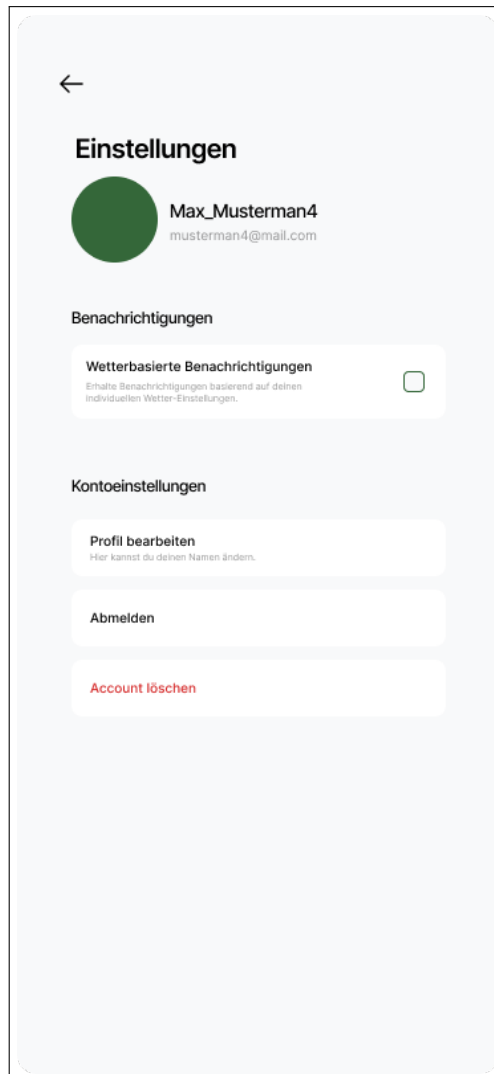


Abbildung 1.6: [Einstellungen](#): Verwaltung von Benachrichtigungen und Kontodetails

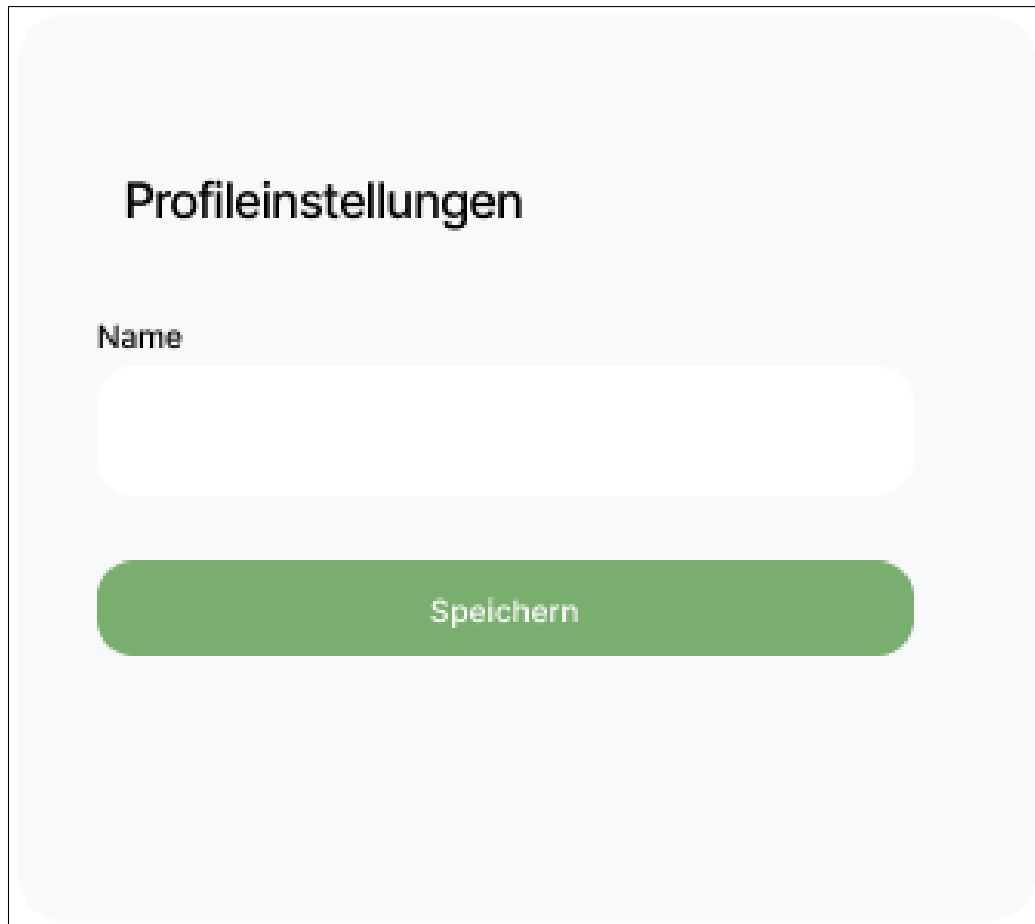


Abbildung 1.7: [Profileinstellungen](#): Menü zur Änderung des Benutzernamens

Push-Benachrichtigungen (siehe Abbildung 1.8) informieren den Nutzer bei Inaktivität sowie bei erreichten Streak-Meilensteinen. Das Home-Screen-Widget (siehe Abbildung 1.9) ermöglicht einen schnellen Überblick über die wöchentliche Streak ohne die App öffnen zu müssen.

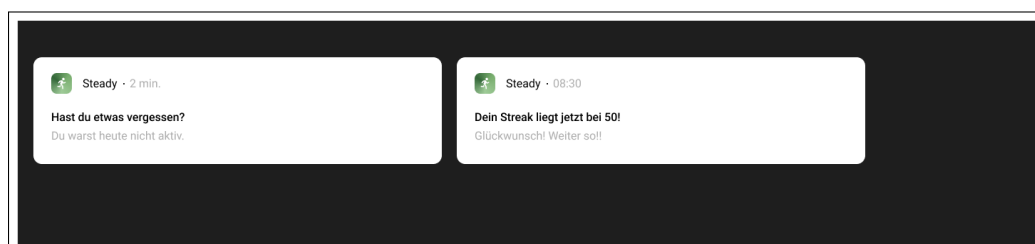


Abbildung 1.8: [Benachrichtigungen](#): Darstellung von Inaktivitäts-Erinnerungen und Streak-Meilensteinen

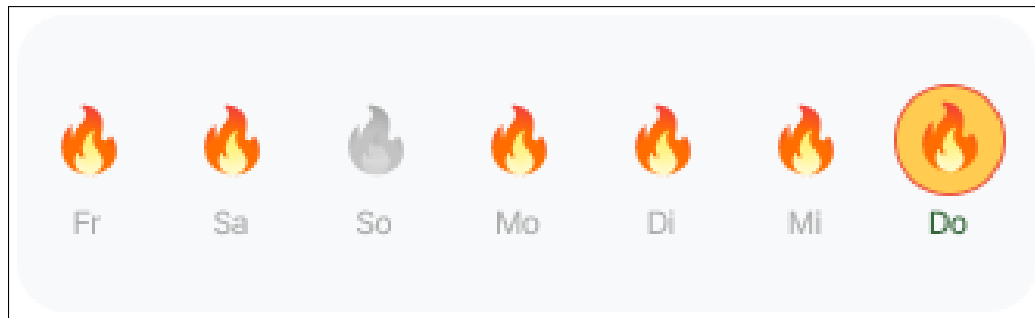


Abbildung 1.9: Streak-Widget: Isolierte Ansicht der wöchentlichen Streak-Visualisierung

1.4 Architektur (konzeptionell + technologisch)

1.4.1 Logische Sicht

Die Anwendung ist in drei Schichten aufgeteilt: UI-Schicht, Logik-Schicht und Datenzugriff. Wie in Abbildung 1.10 dargestellt, kommuniziert die UI-Schicht (Jetpack Compose) ausschließlich über die Logik-Schicht mit dem Datenzugriff. Der Datenzugriff kommuniziert sowohl mit der lokalen Room-Datenbank als auch mit dem Spring Boot Backend und der WeatherAPI über HTTP.

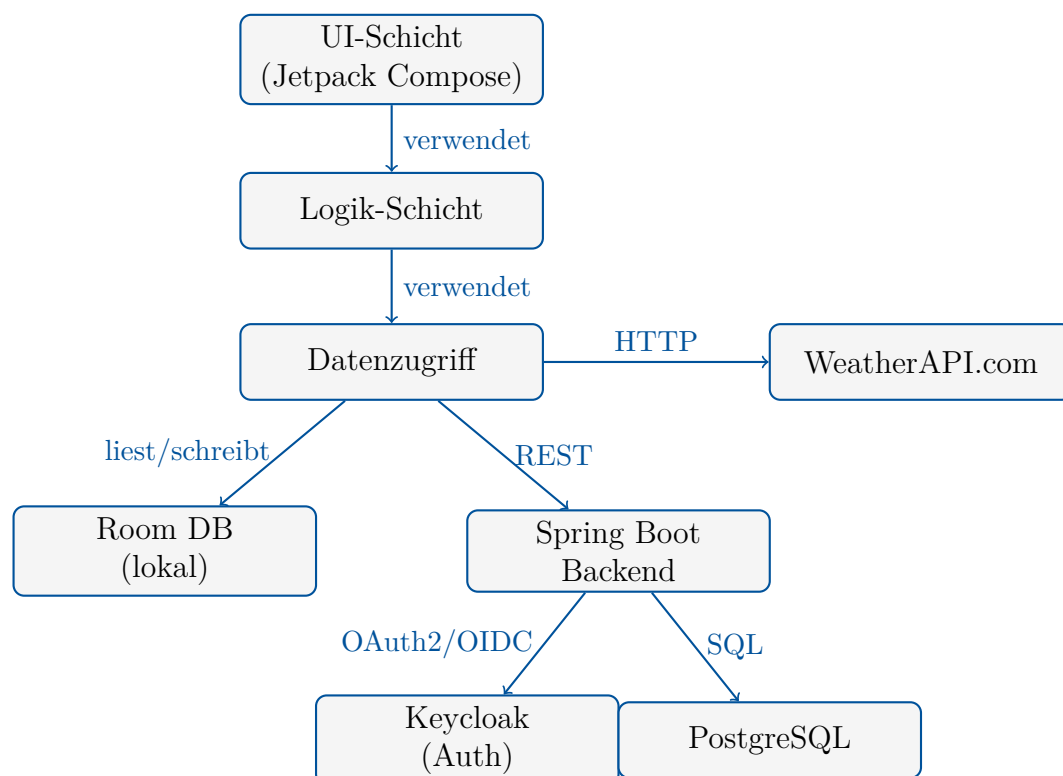


Abbildung 1.10: Logische Architektur – Schichtenmodell

1.4.2 Technischer Stack

Der technische Stack (siehe Tabelle 1.1) umfasst alle eingesetzten Technologien und Bibliotheken der Anwendung. Die Begründung der einzelnen Technologieentscheidungen ist in Abschnitt 1.4.3 beschrieben.

Schicht	Technologie / Bibliothek
Frontend	Android App + Jetpack Compose (Kotlin)
Architektur	Schichtenbasierter Ansatz (UI, Logik, Datenzugriff)
Lokale DB	Room (SQLite) – Offline-Verfügbarkeit
API-Client	Retrofit 2 – REST-Kommunikation
Backend	Spring Boot – REST API und Datenverwaltung
Authentifizierung	Keycloak
Datenbank	PostgreSQL
Wetter-API	WeatherAPI.com – wetterbasierte Erinnerungen
Widgets	Glance API (Jetpack)
Versionskontrolle	GitHub ¹
Build-System (Android)	Gradle (Kotlin DSL)
Build-System (Backend)	Maven

Tabelle 1.1: Technischer Stack des Projekts

1.4.3 Begründung der Technologiewahl

Die Auswahl der eingesetzten Technologien basiert sowohl auf Vorgaben des Auftraggebers als auch auf technischen und wirtschaftlichen Überlegungen des Teams.

Keycloak wurde als Authentifizierungslösung gewählt, da es vom Auftraggeber pep.digital und der Hochschule vorgegeben wurde und den OAuth2/OIDC-Standard vollständig implementiert.

PostgreSQL wird serverseitig als primäre Datenbank eingesetzt, da Nutzerdaten und Aktivitäten zentral gespeichert werden müssen, um geräteübergreifenden Zugriff zu ermöglichen. Room dient lediglich als lokaler Cache für Offline-Verfügbarkeit und als Ergänzung zu PostgreSQL.

Spring Boot wurde als Backend-Framework gewählt, da es vorgegeben wurde und für die Tracking-App mit den anderen eingesetzten Technologien kompatibel ist.

Jetpack Compose ist das moderne UI-Framework für Android und wird von Google als offizieller Nachfolger des klassischen XML-basierten View-Systems empfohlen. Es wurde vom Auftraggeber vorgegeben und ermöglicht eine deklarative UI-Entwicklung,

¹Projekt-Repository: <https://github.com/hse-classes/ss26-aktivitatstracker>

bei der die Oberfläche direkt den aktuellen Zustand der Anwendung widerspiegelt. Gegenüber dem klassischen View-System entfällt unnötiger Overhead im Code.

Glance API wurde für die Umsetzung der Widgets gewählt, da sie eine moderne und deklarative Entwicklung von Android-Widgets ermöglicht. Die API reduziert den Implementierungsaufwand und verbessert die Wartbarkeit des Codes. Zudem passt Glance gut zum Jetpack-/Compose-Ansatz und eignet sich besonders für einfache, schnell zugängliche Informationen wie Aktivitäten und Streaks.

Retrofit ist bekannt dafür, Netzwerkanfragen für Android- und Java-Entwicklungen in wenigen Zeilen Code abzubilden. Es wurde ausgewählt, um die Verwaltung von REST-Calls zu vereinfachen und den Code lesbarer zu gestalten.

WeatherAPI.com wurde ausgewählt, da der kostenlose Tarif mit 100.000 Anfragen pro Monat für den Projektumfang ausreichend ist. Die Integration erfolgt via Retrofit, wobei der API-Key sicher im Backend verwaltet wird. Ein Vorteil ist die Rückgabe der Daten als strukturiertes JSON, was eine direkte Verarbeitung im Projekt ermöglicht (siehe Quelle [2]).

Room wurde gewählt, da es als offizieller Google-Standard gut dokumentiert ist und Datenbankfehler bereits beim Kompilieren erkennt. Die direkte Umwandlung von Datenbankeinträgen in Kotlin-Objekte reduziert den Implementierungsaufwand und verhindert typische Laufzeitfehler.

1.5 Projektmanagement

1.5.1 Ziel des Projektmanagements

Ziel ist eine strukturierte und nachvollziehbare Projektdurchführung in agilen Ein-Wochen-Sprints. Durch regelmäßige Meetings und konsequentes Issue-Tracking soll sichergestellt werden, dass Termine eingehalten und Fortschritte transparent dokumentiert werden.

1.5.2 Arbeitsweise

Das Team arbeitet nach einem Issue-basierten Ansatz ohne fest zugewiesene Rollen. Aufgaben werden als Issues im GitHub-Repository angelegt und vom Team untereinander aufgeteilt. Dies fördert flexible Zusammenarbeit und gleichmäßige Beteiligung aller Mitglieder. Das Projekt-Repository ist unter folgendem Link erreichbar: <https://github.com/hse-classes/ss26-aktivitatstracker>

1.5.3 Kommunikations- und Organisationstools

Zur Koordination und Kommunikation im Team werden die in Tabelle 1.2 aufgeführten Tools eingesetzt.

Tool	Verwendungszweck
GitHub	Versionskontrolle, Issue-Tracking, Kanban-Board
Discord	Team-Kommunikation
Figma	UI-Design, Prototypen
Overleaf	Dokumentierung
Microsoft Teams	Meetings mit Betreuer und Auftraggeber

Tabelle 1.2: Kommunikations- und Organisationstools

1.6 Aufwandsschätzung

Die folgende Tabelle (siehe Tabelle 1.6) zeigt die geschätzten Aufwände pro Issue. Die Schätzung basiert auf **Tagen (T)**, wobei gilt: **1 Tag = 8 Stunden**, **0,5 Tage = 4 Stunden**. Das Projekt ist in **Ein-Wochen-Sprints** gegliedert. Die Schätzungen wurden gemeinsam im Team festgelegt und orientieren sich an den angelegten Issues im GitHub-Repository.

Issue	Beschreibung	Gesch. Aufwand	Tats. Aufwand
#1	Grundgerüst Frontend/App	0,5T	–
#2	UI für Aktivitätseintrag	1T	–
#3	Lokale Aktivitätseinträge in DB speichern	1T	–
#4	Lokale DB aufsetzen	1T	–
#5	Grundgerüst Backend und Remote-DB	1T	–
#6	Datenmodell erstellen	0,5T	–

Hinweis: Die Schätzungen sind Planungswerte. Tatsächliche Aufwände werden im GitHub per Time-Tracking festgehalten.

Anhang

Das folgende Glossar erläutert die in diesem Dokument verwendeten Fachbegriffe.

Begriff	Erklärung
MVP	Minimum Viable Product – minimaler funktionsfähiger Prototyp
IAM	Identity and Access Management – Verwaltung von Identitäten und Zugriffsrechten
OAuth2	Offenes Autorisierungsprotokoll für sicheren Zugriff auf Ressourcen
OIDC	OpenID Connect – Authentifizierungsschicht auf Basis von OAuth2
REST	Representational State Transfer – Architekturstil für Webservices
API	Application Programming Interface – Schnittstelle zur Kommunikation zwischen Systemen
Room	Googles offizielle Bibliothek für lokale Datenbanken auf Android, basierend auf SQLite
Glance	Jetpack-Bibliothek zur deklarativen Entwicklung von Android Home-Screen-Widgets

Quellen

1. Figma-Entwürfe, erstellt von Lionel Toussoumsnaba, 2026. <https://www.figma.com/design/QJRfYFM1shPvySzpfwocuo/Activity-Tracker?node-id=0-1&t=z3Sr26iHMMWxgfBu-1>
2. WeatherAPI.com. <https://www.weatherapi.com>
3. Projekt-Repository auf GitHub. <https://github.com/hse-classes/ss26-aktivitatstracker>